

COMP3027J 课程

软件架构

软件架构描述

邓永健

北京工业大学计算机学院数据

采矿与安全实验室 (DMS 实验室)



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

大纲

1. 软件架构描述

2. 软件架构建模



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

大纲

1. 软件架构描述



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

什么是软件架构描述

软件架构由某种形式的结构元素组成，即组件的集合。包括处理组件、数据组件和连接组件。

处理组件负责处理数据。

数据组件代表经过处理的信息。

连接组件负责组合和连接不同的组件。



Rational Rose



北京工业大学
BEIHANG UNIVERSITY OF TECHNOLOGY



什么是软件架构描述

软件架构是正在开发的系统的蓝图。

它明确了工作分配。

它是质量属性的主要载体。

这是早期分析的最佳工具。

这对于部署后的维护和挖掘至关重要。

描述软件系统结构是创建架构过程中**最为重要的一步**。

无论是现在还是 20 年后，软件架构文档都能体现软件架构的设计理念。

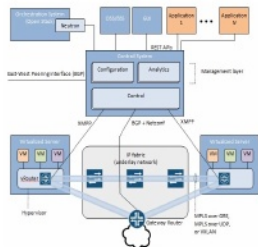


软件架构描述 - 功能

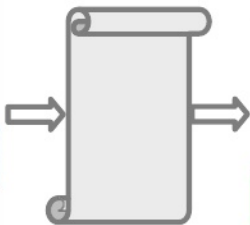
描述软件架构是理解架构设计理念以及交流如何利用架构来指导系统构建的基础。

如果你无法理解建筑，又该如何去运用它呢？

如果无法交流，又如何理解呢？



**Design
Decisions**



**Software
Architecture
Description**



Communication



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 何时进行

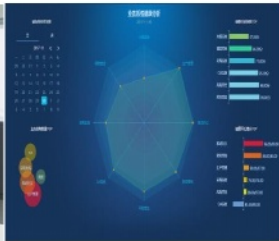
当我们浏览公司的网站、与客户互动或进行软件架构评估时，我们需要获取一份有关软件架构的描述文档作为沟通的基础。



网上冲浪



通信



评估



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 方法

行业中使用的**传统描述方法**存在一定的局限性，而专业的软件架构描述文档可以解决这一问题。

在工业领域，通常采用的描述方法或问题表述方式为：

“使用 UML。”

“框架和线条。”

- 除了我的“玫瑰”类图之外，我还需要什么？

“自然语言不是很好。”

“如何记录变更的信息？”



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 原则

建立描述性文件的基本原则

1. 从读者¹的角度来写。
2. 避免不必要的重复。
3. 避免歧义。
4. 采用标准的组织结构。
5. 记录原因。
6. 保持文件内容最新，但不要频繁更新。
7. 审查这些文件是否符合要求



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 原则

1. 从读者的角度来写。

读者在阅读这份文件时希望了解什么？

1. 让信息易于查找！
2. 您的读者会感激您的努力，并且更有可能阅读您的文档。

- 为方便作者而做的文件标记：

1. 意识流：作者脑海中事件发生的顺序
2. 执行流程：计算机中事件发生的顺序



软件架构描述 - 原则

2. 避免不必要的重复。

每一条信息都应准确记录。

这使得文档更易于使用和修改。

重复常常会造成混淆，因为信息以略有不同的方式被重复，而你不知道哪一种才是正确的？



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 原则

3. 避免歧义。

文件用于交流信息和想法。如果造成误解，该文件将无效。

1. 精确定义的符号/语言有助于避免歧义。
2. 编纂词典有助于避免歧义。

- 如果该文件使用的是图形语言

1. 需要创建一个图例
2. 图例需要正式地定义或给出每个符号的含义。



软件架构描述 - 原则

4. 采用标准的组织结构。

建立标准的组织架构，确保文件遵循该架构，并让读者了解其内容。

- 未完成的部分不要留空，请标注 “待完成”。

5. 记录原因。

为什么做出这些设计决策？

记录基础知识可以节省大量时间。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 原则

6. 保持文件内容最新，但无需频繁更新。

不完整、过时、与事实不符以及不符合本组织结构要求的文件不应使用。

更新文档时，选取发展计划中的关键点。

- 采用适合项目的发布策略。

7. 审查这些文件是否符合要求。

- 为实现我们已确定的目标而精心挑选的信息？

- 建筑文档的读者都有谁？

他们使用这份文件的目的是什么？



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

大纲

2. 软件架构建模



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 视图

“view” 的定义。

“视图”这一概念为我们提供了描述建筑的原则：

1. 记录相关意见
2. 然后添加适用于多个视图的信息
3. 因此将想法联系在一起

- **架构元素** - 软件或硬件中实际存在的元素集合

- **视图** - 由系统利益相关者编写和阅读的、对一组连贯的架构元素的表述



软件架构描述 - 视图

“view” 的定义。

视图是一组架构元素及其关系的表示形式。

但并非所有的建筑元素都有涵盖。

该视图在描述架构时会绑定感兴趣的元素类型和关系类型，并将其展示出来。



软件架构描述 - 视图

“view” 的定义。

视图是一种复杂的管理方法。

每个视图都回答了有关架构的不同问题。

1. 主要的执行单元和数据存储是什么？
2. 其他软件是如何使用这款软件的？
3. 数据是如何在系统中流动的？
4. 如何将软件部署到硬件上？



软件架构描述 - 视图

视图类型。

- 展开视图 - 显示与“**关联子模块**”关系相关的模块
- 使用视图 - 显示与“**使用**”关系相关的模块（即一个模块使用另一个模块提供的服务）
- 层次视图 - 展示了按相关且连贯的**功能划分**的**模块组**。每个组代表整体结构中的一层。
- 类/泛化视图 - 显示被称为**类**的模块，这些模块通过“继承”或“实例”关系相互关联



软件架构描述 - 视图

视图类型。

- **进程视图** - 展示通过通信、同步和互斥操作相互连接的进程或线程
- **并发视图** - 展示组件和连接器，其中连接器代表“逻辑线程”
- **共享数据（存储库）视图** - 展示创建、存储和访问持久数据的组件和连接器
- **客户端 - 服务器视图** - 展示了相互协作的客户端和服务端以及它们之间的连接器（即它们共享的协议和消息）



软件架构描述 - 视图

视图类型。

- **部署视图** - 展示软件元素及其在硬件和通信元素中的分配情况
- **实现视图** - 展示开发、集成和配置控制环境中的软件元素及其与文件结构的映射关系
- **工作分配视图** - 展示模块以及它们是如何分配给负责实现和集成这些模块的开发团队的



软件架构描述 - 视图

基于视图的架构建模规范。

软件应用程序架构的规范被称为**架构描述**。

在 IEEE 1471 的术语中，系统具有架构，该架构由架构规范来描述。

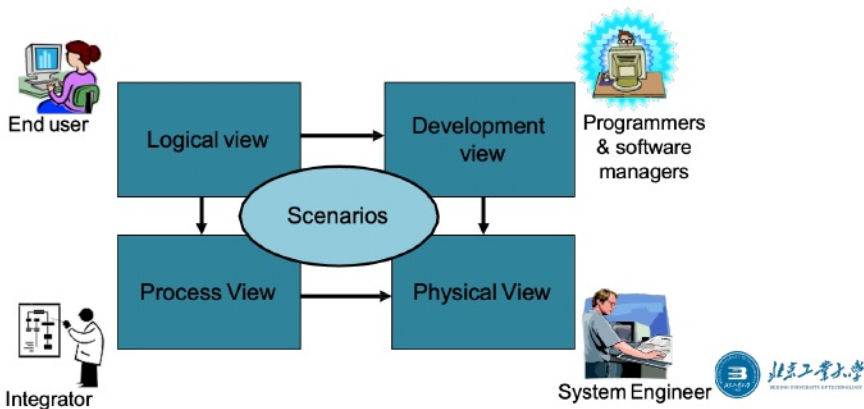
1. 该架构描述选择了一个或多个视角。
2. 每个视角都包含一个或多个相关视图。
3. 一个视图由一个或多个模型组成，并专注于单一的视角。
4. 架构描述由一个或多个视图组成。



软件架构描述 - 视图

基于视图的架构建模规范 - 4+1 视图模型。

1995 年, Rational 公司的克鲁奇滕 (Kruchten) 提出了用于架构描述的“4 + 1”模型。该模型基于佩里 (Perry) 和沃尔夫 (Wolf) 以及贝里·博姆 (Berry Boehm) 提出的架构定义。



软件架构描述 - 视图

基于视图的架构建模规范 - 4+1 视图模型。

- **逻辑视图**：支持行为需求。其关键抽象是对象或对象类。
- **进程视图**：处理并发和分布式问题。将线程映射到对象。
- **开发视图**：组织软件模块、库、子系统和开发单元。
- **物理视图**：将其他元素映射到处理和通信节点。
- **用例视图**：通过将其他视图映射到重要的用例（这些用例被称为场景）来说明架构，这构成了第五种视图。



软件架构描述 - 视图

基于视图的架构建模规范 - 其他。

西门子 4 视图：概念视图模块；互连视图、执行视图；代码视图。

赫茨姆与西姆斯：技术架构；应用架构；项目管理架构；功能架构

降低软件成本的视角：模块分解视图；过程视图；使用视图。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 视图

如何选择视图？

许多作者和方法都指定了标准的一组视图，但每个视图都有一定的成本和相应的收益。因此，我们需要一种选择视图的方法。

- 创建表单。

1. 行：列出与兴趣相关的系统架构。
2. 列：列出了一组可应用于所记录架构的样式。
3. 如果感兴趣的系统架构 x 需要视图 y ，请勾选 (x, y) 盒子。

适当合并视图以减少数量。

根据需求确定观点的优先级（某些利益相关者的意见可能具有更大的权重）。

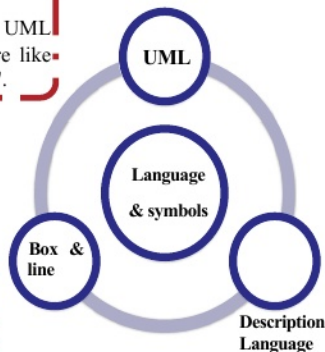


软件架构描述 - 统一建模语言 (UML)

UML 与其他方法相比, 有哪些优势?

1. It is not intended to document architectural information.
2. In fact the standard language UML 2.0 is better, it has architecture like "components" and "connectors".

1. Always use legends
2. Advantages: Flexibility.
3. Disadvantages: vague, no tool support



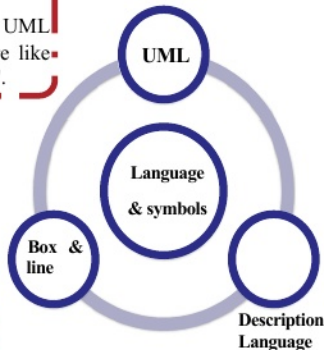
1. Subject of much research in the 1990s; not used much in practice
2. Architecture Description Language (ADL) recently became an IEEE standard.



软件架构描述 - 统一建模语言 (UML)

UML 与其他方法相比, 有哪些优势?

1. It is not intended to document architectural information.
2. In fact the standard language UML 2.0 is better, it has architecture like "components" and "connectors".



1. Always use legends
2. Advantages: Flexibility.
3. Disadvantages: vague, no tool support

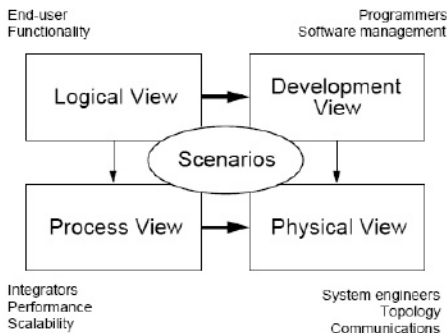
1. Subject of much research in the 1990s; not used much in practice
2. Architecture Description Language (ADL) recently became an IEEE standard.



软件架构描述 - 统一建模语言 (UML)

4 + 1 视图模型

“4+1”视图模型从**逻辑视图**、**进程视图**、**物理视图**、**开发视图**和**场景视图**这五个不同的**视角**来描述软件架构。每个视图仅关注系统的某一方面，只有这五个视图结合起来才能反映系统软件架构的全部内容。

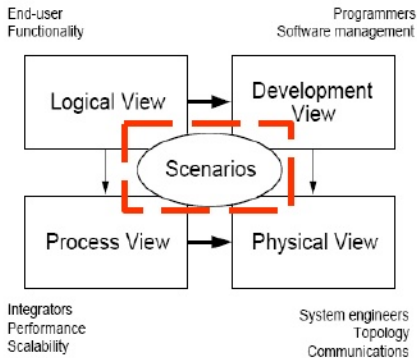


软件架构描述 - 统一建模语言 (UML)

4 + 1 视图模型

- 用例视图 (场景)

1. 从外部世界的角度描述所建模系统的功能。
2. 这种视图用于描述系统应执行的操作。所有其他视图都依赖于用例视图 (场景) 来引导, 这就是该模型被称为“4+1”的原因。
3. 这种视图通常包含用例图、描述图和概览图。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

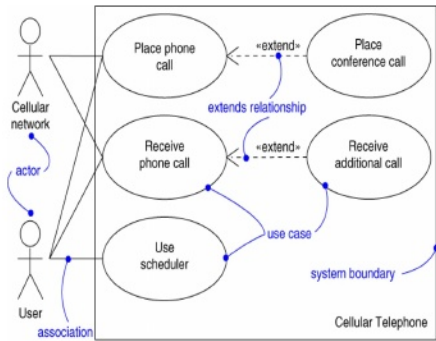
软件架构描述 - 统一建模语言 (UML)

用例图

用例是一种很有价值的工具，有助于理解系统的功能需求，是对系统所提供功能的描述。

1. 用于展示多个角色以及这些角色与系统所提供的用例之间的关联关系。

2. 重要的是要记住，用例代表的是系统的外部视图；因此，不要期望用例与系统内部的类之间存在任何关联。



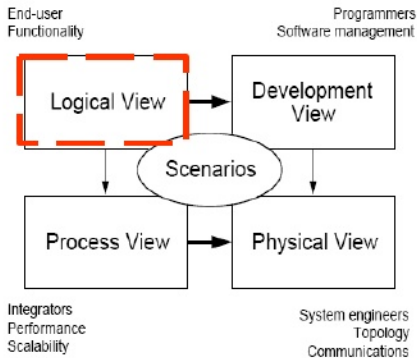
软件架构描述 - 统一建模语言 (UML)

4 + 1 视图模型

- 逻辑视图

1. 一种对系统各部分进行抽象描述的方式。用于对系统的组件进行建模以及描述组件之间的相互作用。

2. 通常包括类图、对象图、状态图和通信图。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

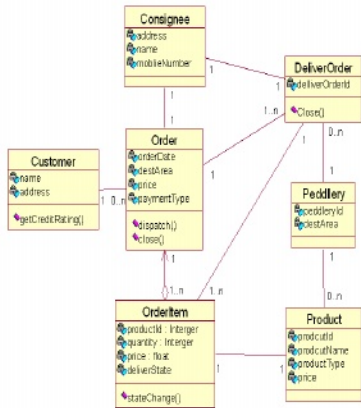
软件架构描述 - 统一建模语言 (UML)

类图

类图表示系统中的类以及类之间的关系。它是对系统静态结构的一种描述。

类图的问题在于它们极其多样，以下是一些使用它们的小贴士：

2. 概念类图在探索业务语言时非常有用。
3. 不要事事都画模型；与其画很多被遗忘的、过时的模型，不如用几个最新的图表。

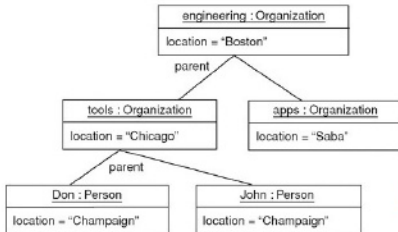
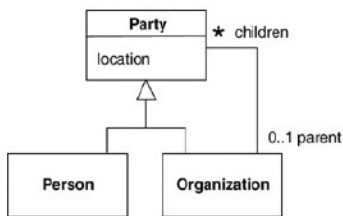


软件架构描述 - 统一建模语言 (UML)

对象图

对象图是系统中对象在某一时刻的状态快照。由于它展示的是实例而非类，因此通常也被称为实例图。

对象图有助于展示相互关联的对象实例。在很多情况下，尽管结构可以通过类图精确地定义，但理解起来仍有难度。在这些情况下，几个对象图示例会大有帮助。

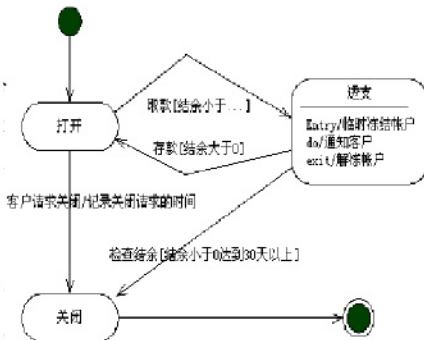


软件架构描述 - 统一建模语言 (UML)

状态图

状态图描述了一个类的对象的所有可能状态以及当事件发生时状态的转换条件。通常，状态图是对类图的补充。

1. 擅长描述对象在多种用例中的行为，不太擅长描述涉及众多对象协作的行为。
 2. 使用状态图时，不要试图为系统中的每个对象都绘制状态图。仅对那些明显具有特殊行为的对象使用状态图，这有助于您理解其工作原理。用户界面和控制对象是使用状态图描述的最常见对象，因为它们具有多种类型的状态。
- 行为。

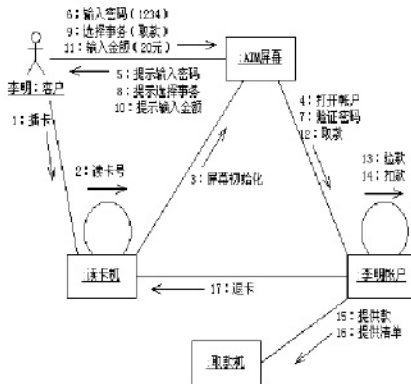


软件架构描述 - 统一建模语言 (UML)

通信图

通信图是一种交互图，它着重强调发送和接收消息的对象之间的组织结构。通信图展示了一系列对象、对象之间的连接以及对象之间发送和接收的消息。

1. 描述对象之间的协作关系时，协作图（通信图）与序列图类似，展示了对象之间的动态协作关系。



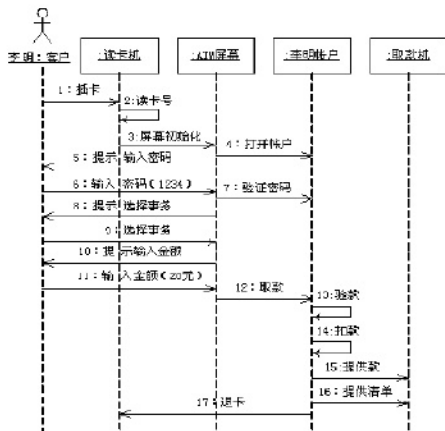
软件架构描述 - 统一建模语言 (UML)

序列图

在软件工程中，序列图是对象交互的一种表示形式。主要用于展示对象之间按其发生顺序排列的一系列交互。

1. 它用于反映多个对象之间的动态协作关系，即这些对象如何随时间相互作用。

2. 通常用于在一个用例中查看多个对象的行为。它更侧重于描述对象之间的协作，不太擅长对它们的行为进行精确定义。

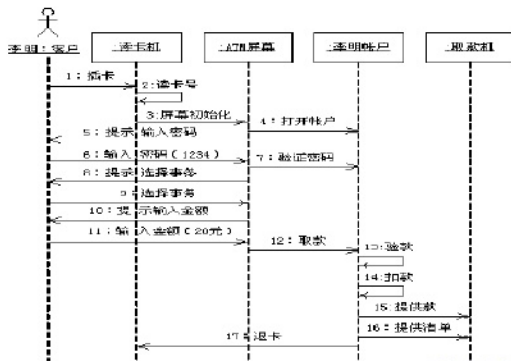


软件架构描述 - 统一建模语言 (UML)

序列图

在软件工程中，序列图是对象交互的一种表示形式。主要用于展示对象之间按其发生顺序排列的一系列交互。

3. 状态图通常用于查看单个对象在多个用例中的行为。通常在查看多个用例或多个线程中的行为时，考虑使用活动图。

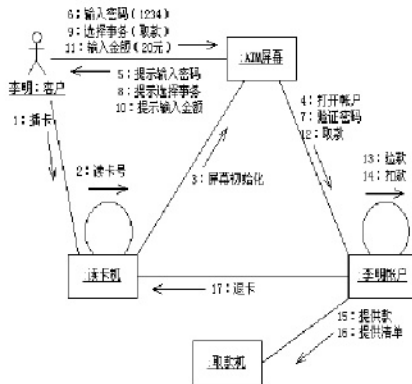


软件架构描述 - 统一建模语言 (UML)

通信图

1. 主要的问题在于何时使用通信图而非更常见的序列图。这在很大程度上取决于个人偏好：有些人更倾向于使用其中一种。

2. 当您想要强调调用顺序时，最好选择序列图；当您想要强调关联时，最好选择通信图。许多人发现通信图在白板上更容易修改，因此是探索替代方案的好方法。

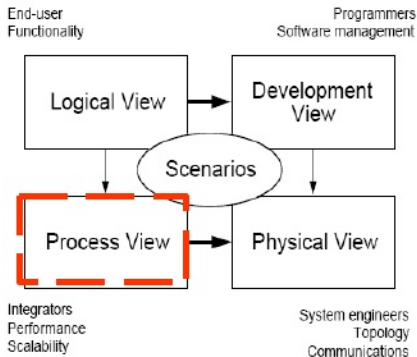


软件架构描述 - 统一建模语言 (UML)

4 + 1 视图模型

- 过程观

1. 描述系统中的流程。这种情况在可视化系统中必须发生的情况时特别有用。
2. 这种视图通常包含一个活动图。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

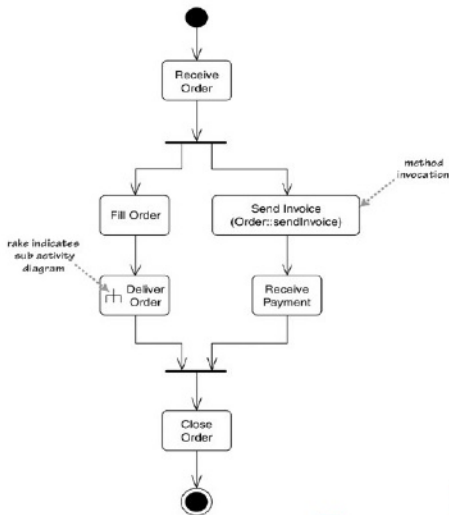
软件架构描述 - 统一建模语言 (UML)

活动图

描述为满足用例需求而要执行的活动以及活动之间的约束关系，有助于识别并行活动。

1. 其巨大的优势在于支持并鼓励并行行为。这使其成为工作流和流程建模的绝佳工具，UML 2.0 的很多推进实际上都来自工作流领域的相关人员。

2. 原则上，可以利用分支和合并的优势来描述并发程序的并行算法。



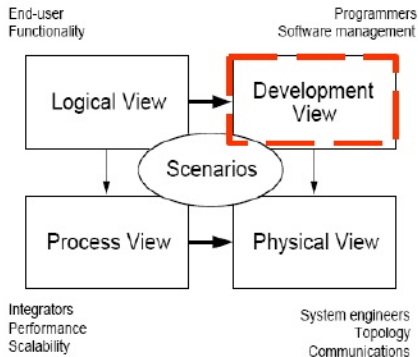
软件架构描述 - 统一建模语言 (UML)

4 + 1 视图模型

- 发展观点

1. 描述系统中各部分是如何被组织成模块和组件的。这对于管理系统架构中的各层非常有用。

2. 这种视图通常包含包图和组件图。



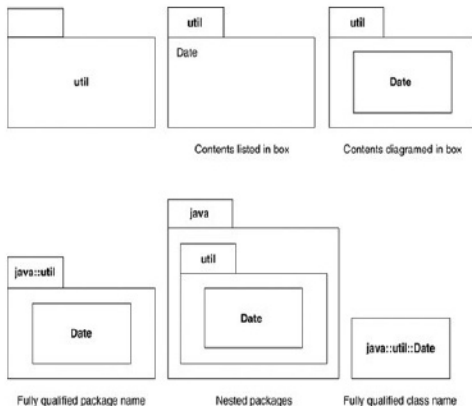
北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 统一建模语言 (UML)

包图

包是用类似于文件夹的符号表示的 UML 模型元素的组合，它允许将 UML 中的任何结构及其元素分组为更高级别的单元。

1. 包可以在 UML 的任何部分使用。
2. 系统中的每个元素只能属于一个包，而且一个包可以嵌套在另一个包内。
3. 使用包图将相关元素分组到一个系统中
4. 一个包可以包含卫星包、图表或单个元素。



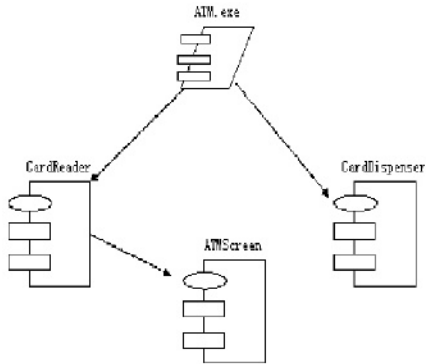
北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 统一建模语言 (UML)

组件图

组件图描述了代码组件的物理结构以及组件之间的依赖关系。将系统划分为组件，并通过接口或组件向更低层级结构的细分来展示它们之间的相互关系。

1. 其主要目的是展示系统组件之间的结构关系。
2. 适用于不同群体的实用交流工具。

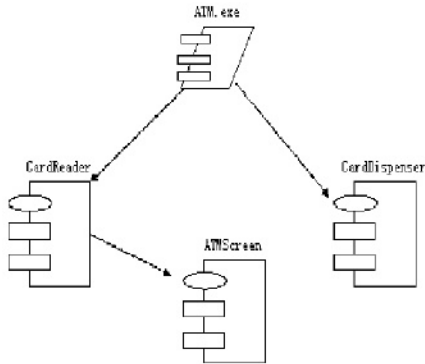


软件架构描述 - 统一建模语言 (UML)

组件图

组件图描述了代码组件的物理结构以及组件之间的依赖关系。将系统划分为组件，并通过接口或组件向更低层级结构的细分来展示它们之间的相互关系。

3. 在基于组件的开发 (CBD) 中, 组件图向架构师提供了一种自然的形式来开始对解决方案进行建模, 并且允许架构师验证系统所需的功
能是否由组件实现。



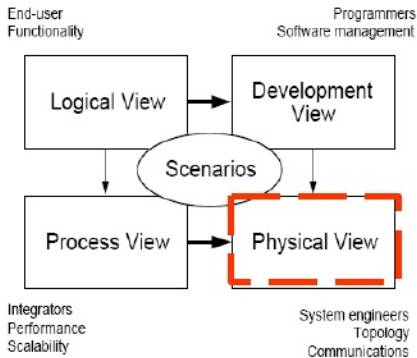
软件架构描述 - 统一建模语言 (UML)

4 + 1 视图模型

- 物理视图

1. 描述在前三个视图中所描述的系统设计是如何实现为一组现实世界中的实体的。此视图中的图表展示了抽象部分是如何映射到最终部署的系统中的。

2. 这种视图通常包含一个部署图。

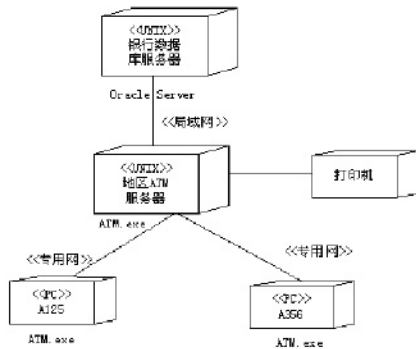


北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

软件架构描述 - 统一建模语言 (UML)

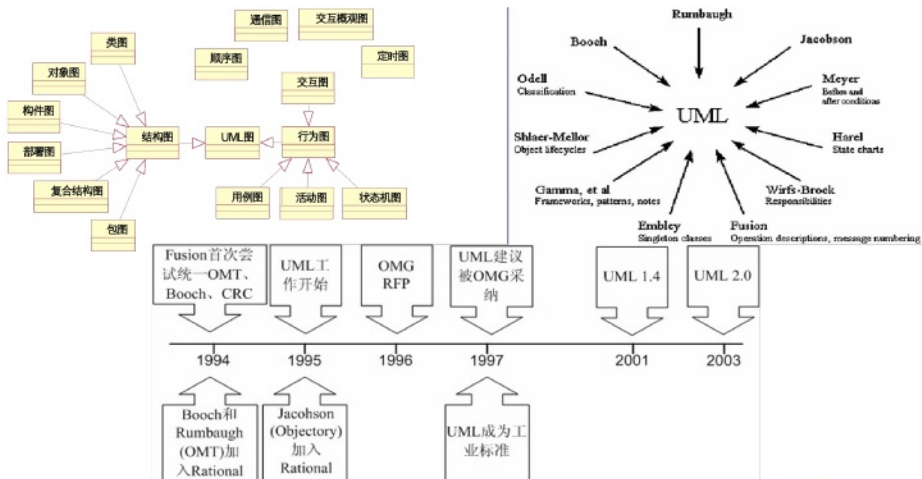
部署图

部署图定义了系统中软件和硬件的物理架构。描述了运行时的硬件节点以及在这些节点上运行的软件组件的静态视图。展示了系统硬件、安装在硬件上的软件以及用于连接异构机器的中间件。



1. 从部署图中，您可以了解软件和硬件组件之间的物理关系以及组件在处理节点上的分布情况。
2. 使用部署图来展示运行时系统的结构，同时传达构成应用程序的硬件和软件元素是如何配置和部署的。

软件架构描述 - 统一建模语言 (UML)



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

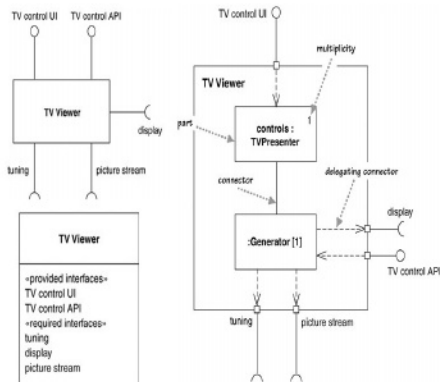
软件架构描述 - 统一建模语言 (UML)

组合图

组合结构是 UML 2 的一个新特性，即能够将类分层分解为内部结构。这可以将一个复杂对象分解为多个部分。

1. 理解包与组合结构之间差异的一个好方法是：包是一种编译时的分组，而组合结构则表现为运行时的分组。

2. 它非常适合展示组件以及组件如何分解为各个部分；因此，在组件图中使用了许多符号。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

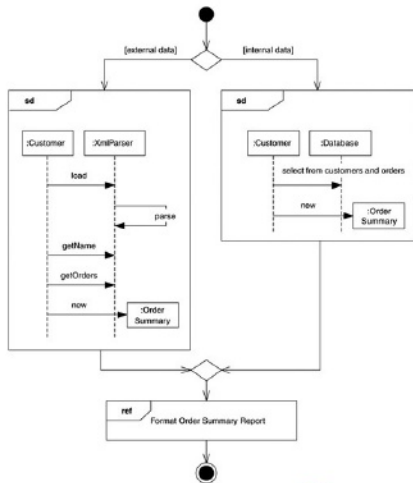
软件架构描述 - 统一建模语言 (UML)

交互图

交互图是活动图和序列图的结合。

您可以将其视为活动图，其中的活动被小型序列图所取代，或者将其视为用活动图符号分解的序列图，以展示控制流。

1. 交互概览图在绘制草图时更为适用。首先通过活动图对业务流程进行建模，然后细化一些关键且复杂度较低的活动节点，并使用序列图来表示其对象之间的流程。

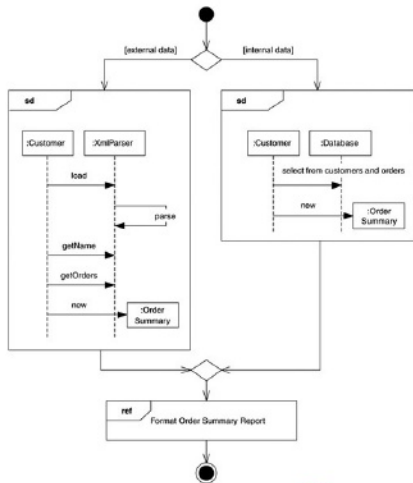


软件架构描述 - 统一建模语言 (UML)

交互图

交互图是活动图和序列图的结合。您可以将其视为活动图，其中的活动被小型序列图所取代，或者将其视为用活动图符号分解的序列图，以展示控制流。

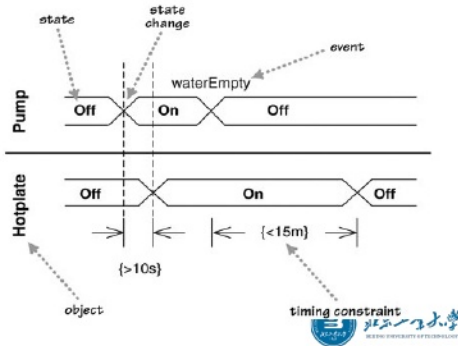
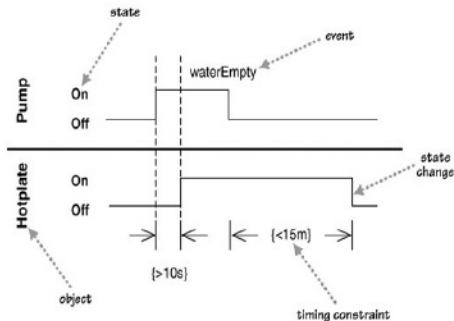
2. 不要盲目使用交互式概览图。对于较大的场景而言，这不是一个好的选择，并且会极大地降低模型的可读性。



软件架构描述 - 统一建模语言 (UML)

时序图

序列图是另一种交互图的形式，其重点在于时间约束。通常用于描述单个对象或多个对象的情况。对于展示不同对象状态变化之间的时间约束非常有用。



面向对象建模语言

面向对象的建模语言大约在 20 世纪 70 年代中期至 80 年代末期之间开始出现，当时的方法论专家们提出了这些语言。

在 1989 年至 1994 年期间，面向对象的模型方法数量从不足 10 种增加到了 50 多种。

格雷迪·布奇的布奇方法——*理智*软件公司。

伊瓦尔·雅各布森（Ivar Jacobson）的面向对象软件工程（OOSE）——对象论（*Objectory*）。

- 詹姆斯·鲁姆巴赫（James Rumbaugh）的对象建模技术（OMT）——*通用*电气公司



面向对象建模语言

简单来说:

- 在项目的规划和构建阶段, Booch 方法尤其具有表现力。
- OOSE 为业务工程和需求分析提供了出色的支持。

OMT-2 对于数据密集型信息系统的分析具有很强的表达能力。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

定义

统一建模语言（UML）是一种用于**可视化**、指定、构建和**文档化**软件密集型系统构件的语言。



建模元素

关系

可扩展性

机制图



建模元素

结构元素

类、接口、协作、用例、组件、节点。

行为要素

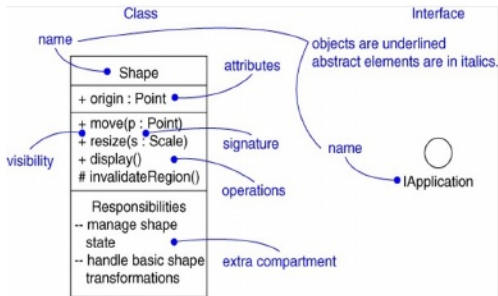
- 互动
- 状态机。

分组元素

- 包，子系统。

其他元素

- 注意。



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY

关系

Dependency(依赖)



Association(关联)



Aggregation(聚合)



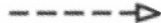
Composition(组合)



Generalization(泛化)



Realization(实现)



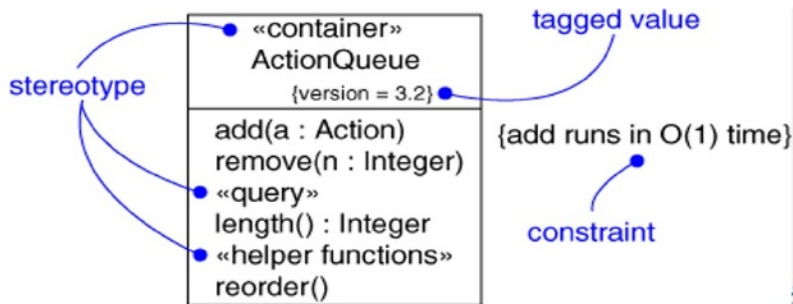
北京工业大学
BEIHANG UNIVERSITY OF TECHNOLOGY

可扩展性机制

刻板印象（构造性）

标记值

约束



图表是对模型的一种展示。

从特定利益相关者的角度呈现

- 提供了系统的部分表示
- 在语义上与其他观点一致

在统一建模语言（UML）中，有 13 种标准图。

- 静态视图：用例、类、对象、组件、部署、包、组合结构

动态视图：序列图、通信图、状态机图、活动图、时序图、交互概览图



谢谢你!

*Documenting Software
Architectures:
Views and Beyond*

Paul Clements	James Ivers
Felix Bachmann	Reed Little
Len Bass	Robert Nord
David Garlan	Judith Stafford

Published by Addison-Wesley in
September 2002 as part of the
SEI Series in Software Engineering



北京工业大学
BEIJING UNIVERSITY OF TECHNOLOGY